To Forge or Not to Forge: Predicting Task-Level Fine-Tuning Gains from Ten-Example Pilots

Anonymous Author(s)
Affiliation
Address
email

Abstract

Fine-tuning a small specialist model is the default path to cheap task-specific capability, yet the upstream decision — is this task worth fine-tuning at all? — is still made by intuition or by paying for the full training run. We ask whether ten labeled examples can answer it in advance. Under a single frozen protocol (Qwen2.5-1.5B-Instruct, fixed LoRA recipe and budgets, greedy decoding, execution-based verifiers), we measure 61 tasks — 11 anchored in the small-model post-training literature and 50 drawn by a pre-registered filter from Super-NaturalInstructions — pairing a ten-example pilot with a full fine-tune whose measured gain serves as ground truth. All analyses were pre-registered. We find: (1) fine-tuning helps far more often than not — 53 of 61 tasks (87%) show positive full-training gains, making "forge" the empirical default in this regime; (2) pilot signals significantly rank gain magnitude (Spearman 0.755, 95% CI [0.60, 0.86]), enabling budget allocation by ranking; (3) the pre-registered binary go/no-go endpoint is not established (leave-one-task-out AUC 0.755, bootstrap CI [0.50, 0.93]), a power limit rooted in the empirical scarcity of no-go tasks itself; and (4) description-only features predict below chance (AUC 0.31) — the decision-relevant information exists only in actually running the pilot. We characterize the no-go tail, including negative transfer on heavily post-trained bases (GSM8K -0.23 despite in-domain training), and release the benchmark, protocol, and all sealed artifacts. These sealed measurements and pre-registered analyses were produced by an autonomous discover-pilot-decide-train-validate system; without that system's contribution the 61-task ground-truth table would not exist.

Keywords: fine-tuning, evaluation, small language models

1 Autonomous research result

1.1 Setup

The operational question is whether a ten-example LoRA probe can tell, in advance, whether full fine-tuning of a small specialist will raise greedy pass@1. We test that question under one frozen protocol, with the analysis contract written down before the remaining production runs, and with an honesty clause that a non-significant AUC is published as a negative result.

Protocol. The base is Qwen2.5-1.5B-Instruct [Qwen Team, 2024], weights pinned at birth (`989aa798`…; full sha in the protocol file). LoRA rank 16, $\alpha = 32$, dropout 0.05, on attention and MLP projections; sequence length 4096; per-device batch 2 $\times$ accumulation 8 (effective batch 16); learning rate 1.0×10^{-4} cosine, warmup 0.03; completion-only loss on the chat template. Pilot: 10 examples, 100 steps, seed 20260820. Full: cap 8000, 3 epochs, same seed. Eval: 200-example

slice, seed 20260820, greedy decoding. pass@8 (k=8, temperature 1.0, top-p 0.95) is a signal, never the label. Labels are greedy pass@1; Δ is the difference from the unadapted base. go $\Leftrightarrow \Delta_{\text{full}} > 0$ (strict; zeros are no-go).

Sample. The sealed analysis set is n=61 tasks: 11 anchored in the small-model post-training literature (Table 2) and 50 drawn from Super-NaturalInstructions [Wang et al., 2022] by a pre-registered English-and-size filter, stratified on `Categories[0]`, seed 20260822, with `Source[0]` capped at 3. The intended literature layer was 13; BIRD and introductory APPS produced no go-label and are `excluded for logistics, before any label existed`. The sample contains 53 go and 8 no-go tasks ($53/61 \approx 86.9\%$). Two long-context tasks (TyDiQA, SuperNI task419) entered after a disclosed OOM fallback that keeps effective batch 16 ($\Delta_{\text{full}} + 0.530$ / $+0.505$). ARC-Easy is in-sample as PARTIAL with metrics present.

Predictors and intervals. The primary model is a task-level leave-one-task-out logistic regression on the frozen numeric signal vector, including a pre-declared format-compliance scalar. Interval estimates are task-level bootstrap 95% CIs on the LOTO pairs (y_i, p_i). The secondary endpoint is Spearman $\rho(\Delta_{\text{pilot}}, \Delta_{\text{full}})$. A text-only logistic that never sees a probe — train size, `max_new_tokens`, prompt-style length, eval n — is the no-pilot control. Single-feature logistics are reported alongside. Cluster-level AUCs were pre-declared and are not estimable.

1.2 Results

Four findings, in the frozen abstract order. Every interval below is the sealed task-level bootstrap 95% CI.

Finding 1: forging is the default. Fine-tuning helps far more often than not: 53 of 61 tasks (87%) have $\Delta_{\text{full}} > 0$. Figure 1 is the whole point: the gate $\Delta_{\text{full}} > 0$ is an easy event on this 1.5B protocol. The scarce class is no-go (8 of 61).

Finding 2: pilots rank gain magnitude. Spearman $\rho(\Delta_{\text{pilot}}, \Delta_{\text{full}}) = 0.755$, CI $[0.595, 0.864]$, which excludes 0. Figure 2 is upward-sloping: the probe ranks *how much* full fine-tuning helps. GSM8K and MATH sit in the third quadrant (both deltas negative) and are labeled. Sensitivities (a) drop ARC-Easy and (b) drop EM-floor rows leave Spearman intact: 0.756 $[0.594, 0.873]$ and 0.771 $[0.608, 0.878]$; both CIs exclude 0.

Finding 3: the binary gate is not established. The main LOTO logistic has AUC 0.755 with CI $[0.500, 0.929]$. The interval covers 0.5. Under the pre-registered honesty clause this is a negative result: a 10-example LoRA probe does not significantly classify whether full fine-tuning will raise greedy pass@1. The same verdict holds in the two pre-declared sensitivities, reported next to the main row rather than as afterthoughts (Table 1): (a) drop ARC-Easy, n=60, AUC 0.748 $[0.487, 0.920]$; (b) drop EM-floor rows, n=57, AUC 0.778 $[0.411, 0.985]$. Both CIs cover 0.5. Single-feature Δ_{pilot} is weaker (0.663 $[0.480, 0.825]$); generation length is 0.818 $[0.505, 0.986]$ and covers 0.5 at the edge.

Finding 4: description features run below chance. The text-only control, which never sees a probe or a run metric, has AUC 0.309 $[0.045, 0.622]$ — below chance. Task size and prompt length do not explain go. Decision-relevant information exists only in actually running the pilot.

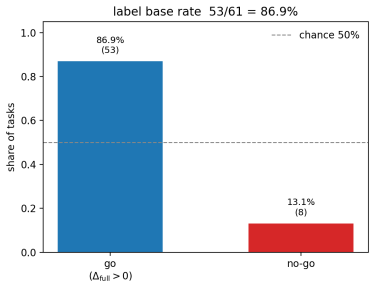

Figure 1: Label base rate $53/61 = 86.9\%$.

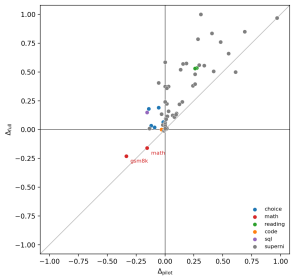

Figure 2: Pilot vs. full gain. GSM8K/MATH labeled in Q3.

Table 1: Primary LOTO AUC and pre-declared sensitivities. Covering 0.5 is a negative under the honesty clause.

Model	n	AUC	95% CI	covers 0.5
Main LOTO logistic	61	0.755	$[0.500, 0.929]$	yes
(a) drop ARC-Easy	60	0.748	$[0.487, 0.920]$	yes
(b) drop EM-floor	57	0.778	$[0.411, 0.985]$	yes
Text-only control	61	0.309	$[0.045, 0.622]$	yes (below chance)

Table 2 reports the 11 literature-layer tasks that are in the sealed sample. BIRD and APPS are out (logistics exclude, no label).

Table 2: Literature-layer tasks in n=61. Display rounding matches the sealed reports; no new analysis.

Task	base	Δ_{pilot}	Δ_{full}
GSM8K	0.690	-0.335	-0.230
WinoGrande	0.560	-0.055	$+0.190$
Spider	0.495	-0.155	$+0.150$
MATH	0.355	-0.155	-0.160
MBPP	0.055	-0.030	$+0.000$
DROP	0.055	$+0.275$	$+0.535$
TyDiQA	0.170	$+0.260$	$+0.530$
ARC-Easy	0.850	-0.120	$+0.035$
ARC-Challenge	0.780	-0.090	$+0.020$
HellaSwag	0.650	-0.140	$+0.180$
PIQA	0.765	-0.015	$+0.065$

1.3 Discussion

The upstream decision — is this task worth fine-tuning? — is still made by intuition or by paying for the full run. A cheap, execution-based probe that ranked tasks would change how small-model post-training budgets are allocated. The measurements above show that such a probe exists as a ranker and does not exist as a binary gate, in a regime where forging is already the default.

The honest reading is split. If the operational question is *whether* to run full fine-tuning, the 10-shot probe is not a significant classifier (Finding 3). If the question is *how large* the full-training gain will be, the same probe ranks tasks (Finding 2). Those two sentences should not be collapsed: the probe is a ranker, not a stoplight.

Finding 1 is the mechanism of Finding 3, not a nuisance. With eight negatives, AUC is underpowered for a hard gate. A classifier that always predicts go is already strong in accuracy and useless in ROC. Missing negatives include two logistics exclusions: BIRD never received metrics; APPS never received Δ_{full}. APPS reached base pass@1 0.0 / pass@8 0.005 before the full-training OOM; that floor is a limitations footnote, not a sample row.

Failure modes in the sample. Format overwrite: GSM8K $\Delta_{\text{full}} = -0.230$ despite in-domain training. The base already emits the dataset's `####` format; full training saturates format while reasoning degrades. MATH sits in the same quadrant ($\Delta_{\text{pilot}} = -0.155$, $\Delta_{\text{full}} = -0.160$). Exact-match floor: four SuperNI rows have $\Delta_{\text{full}} = 0$ because the matcher never hits (`task071`, `task1553`, `task236`, `task455`); sensitivity (b) drops them and the negative gate remains. OOM disclosure: TyDiQA and task419 are in-sample only because of the 1×16 fallback (effective batch still 16). WinoGrande, often treated as a weak-gain control in this size class, shows $\Delta_{\text{full}} = +0.190$ here — a reverse relative to that prior, not a new endpoint.

Related work (lite). Rectified Scaling Law [Lin et al., 2024] predicts pretraining loss versus compute in order to choose which model to fine-tune; we instead ask, for a fixed small model, which *tasks* are worth fine-tuning at all. The Fine-Tuning Trap [Nair and Tao, 2026] documents negative transfer of PEFT on sub-1B mathematical reasoning; our GSM8K/MATH third-quadrant points are a task-level counterpart under a frozen LoRA recipe. Super-NaturalInstructions [Wang et al., 2022] supplies the 50-task slice; CodeS [Li et al., 2024] is the text-to-SQL line that motivates Spider as an anchor.

We do not add features, change the go threshold, or reopen BIRD or APPS. Cluster AUCs stay unreported as estimates: the choice cluster is all-go; the math cluster has n=2.

2 System Design and Meta-Analysis

2.1 Agent and Harness

We ran the study with two coupled pieces: a frozen Python harness that executes one task from package load through a sealed `metrics.json`, and an LLM coding agent that packed those tasks, drove the remote GPU worker, diagnosed failures, ran the pre-registered analysis, and drafted this manuscript. The experimental model inside the harness is Qwen2.5-1.5B-Instruct at a pinned revision; the coding agent's weights are omitted here for double-blind review and are named in the disclosure for the chairs.

The harness is a single-task state machine S0–S6: load and hash-check the package; greedy-evaluate the unadapted base; train a 10-example LoRA probe; evaluate the probe; train the full LoRA; evaluate the full adapter; seal metrics (systems block included). Resume is journal-boundary only: a killed full-training stage restarts at S4 with S0–S3 kept. The agent never edits the protocol file. It may add scripts, isolate a failed run directory, or stop and write a report.

Figure 3 is the loop that produced the sealed n=61 table: a task factory writes frozen packages; a serial queue feeds the S0–S6 runner; gates mark PARTIAL or `over_budget` without inventing labels; incremental then full harvest brings artifacts back; a pre-registered script seals the analysis.

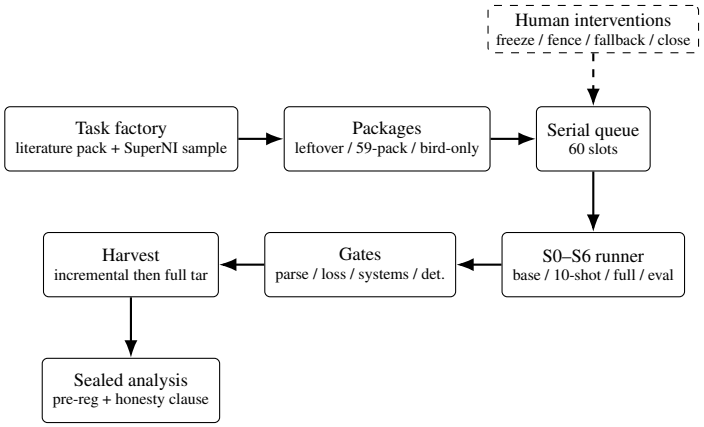

Figure 3: Factory → frozen packages → serial queue → S0–S6 runner → gates → harvest → sealed analysis. Dashed box: human intervention points (freeze, fence, fallback, close).

2.2 Tools

Compute was one 48GB-class GPU worker and one serial queue. The worker ran the harness under a terminal multiplexer; the agent supervised it with scheduled probes of `journal.jsonl`, `STATUS`, and `metrics.json`, because a remote job does not wake the session when it ends. Local tools were git on a single mainline (small commits, no force-push), an append-only ledger, pytest, and SHA-256 manifests. Execution-based verifiers (GSM8K, WinoGrande, Spider, later MATH/MBPP/DROP/TyDiQA/ARC/HellaSwag/PIQA and a SuperNI exact-match family) expose a uniform `verify(output, reference) -> {pass, parsed, note}`; unparseable is `parsed is None`, never a silent wrong. Each verifier has mutation tests: gold must pass, a known-wrong answer must fail, format variants of a correct answer must pass, garbage must unparse. GSM8K accepts `#### 1,000` as 1000 and rejects a hash without a number; WinoGrande will not treat the bare English article "a" as choice A; Spider compares execution multisets and treats a 30 s timeout as fail with the exception type in `note`. The factory pins source revisions at pack time (dataset SHAs in the package `task.json`) and writes a per-package `MANIFEST.sha256`. SuperNI clones were frozen at `55a36563`... before ids were committed; English+size filter left 890 surviving tasks, then the seed-20260822 draw of 50 with a `Source[0]` cap of 3. Code execution is a subprocess with a 10s cap, no network, and a 2 GiB virtual-memory limit.

2.3 Research Loop

The loop was not an open-ended search over architectures. It was discover (pack a frozen task), pilot (10-shot LoRA), decide (record Δ_{pilot} as a signal, not a gate), train (full LoRA), validate (greedy pass@1), then stop. The scientific hypothesis — whether that probe classifies go $\Leftrightarrow \Delta_{\text{full}} > 0$ — was written down before the remaining production runs.

Protocol freeze came first. After three smoke runs the operator stopped a proposed `protocol_v3`: the pilot 10×100 reaching loss $\sim 10^{-5}$ is recitation, and recitation is the probe, not a bug. `protocol_v2.yaml` was never edited after that freeze; any later exception is a run-level flag, not a silent yaml change.

Sample-frame correction came next. An earlier count treated 46 retestable *pool rows* as 46 tasks. Deduped literature-layer tasks are 13 (3 already sealed, 10 new) plus 50 SuperNI tasks drawn by a pre-registered English-and-size filter (seed 20260822, `Source[0]`≤ 3). Intended n=63. A second GPU worker was cancelled; the 60 new slots ran as one serial list. BIRD was packed later as its own package; the 59-task package did not wait. Mini-Dev was never used.

The serial queue started 2026-08-22T09:34Z and drained 60/60 at 2026-08-24T00:22Z on that one GPU (39 h wall). Of those 60 slots, 55 sealed `STATUS=ok` on the 59-task package; the rest were PARTIAL or `over_budget` (TyDiQA and task419 S4 OOM, ARC-Easy unparseable at 5% with metrics, BIRD S0 field error, APPS 3 h S1-only). Three earlier smoke runs (GSM8K, WinoGrande, Spider) already counted as production data under the freeze. A per-task 3 h wall marks `over_budget`; gates run after S6 and do not block the next task. Determinism uses a 200-row rerun on list items 1 and every 10th, otherwise 30 rows. Incremental harvest pulled a 10-directory tar after every tenth GPU completion (adapters omitted). Failed directories are isolated, not deleted. A four-task rerun wave (declared before any AUC) then ran 2026-08-25T06:19Z–13:11Z: BIRD dual-sha checker, TyDiQA and SuperNI task419 with a disclosed 1×16 OOM fallback (effective batch still 16), APPS at an 8 h ops fence. Analysis was sealed the same day at n=61 under the honesty clause. No AUC was computed until that script ran.

2.4 Human Interventions

Human interventions were discrete, dated, and written into the ledger before the next GPU step. We list the ones that changed what the sealed table contains, including the agent's own misreports.

False-negative connectivity. The agent reported that the GPU worker was unreachable. A follow-up over an already-open multiplexed session returned `ok`. The cause was the agent's tool layer refusing the remote-shell call, not a host or authentication failure. Production started after that correction, not after a re-provision. That is an agent error in the log, not a cluster outage.

Procedure slip, then a STOP. Gate 1 (WinoGrande, then Spider) was started before Gate 0's five-eye table had been accepted. The operator halted Spider at the S4 boundary (SIGTERM; S0–S3 kept), kept the WinoGrande directory, and only then passed Gate 0. Spider later resumed from that journal boundary and sealed. The STOP did what a comment in chat cannot: it prevented a protocol decision from being made on a moving GPU.

Format overwrite, not EOS. GSM8K returned $\Delta_{\text{full}} = -0.230$ with unparseable $0/200$ and a deterministic greedy rerun. A truncation/EOS reading does not fit those numbers. Base format-compliance was #### 58 / last-number 142 / none 0; after the probe and after full training the slice was #### 200/200. The operator's Gate 0 ruling — format overwrite with reasoning collapse — is the one we keep. The derived `format_compliance` field was backfilled into metrics and did not bump the protocol version.

Schema lag on BIRD, failed closed. The first BIRD slot died in S0 on `KeyError: sha256`: the checker wanted a single `source.sha256`; the package stored `train_zip_sha256` and `dev_zip_sha256`. No metrics were written. The directory was isolated. The checker was aligned to the dual-sha fields; the bird-only package was not rebuilt. A later slot passed S0, then died of CUDA OOM at the first probe backward pass (S1 pass@1 0.085 / pass@8 0.23; still no `metrics.json`). We did not invent a SQL label to close the queue.

Wall as an ops fence, not a measurement. APPS hit the 3 h cap in S1 with no metrics (`over_budget`). The 8 h rerun was declared before any APPS number existed. It finished S1 at base pass@1 0.0 / pass@8 0.005 and died of CUDA OOM at full-training step 22/495 — still no Δ_{full}. The floor is a missing-negative footnote, not a row.

OOM fallback, two tasks only. TyDiQA and SuperNI task419 died of CUDA OOM in S4 on the first pass. A run-level 1×16 fallback (effective batch 16, yaml untouched, flag in STATUS and metrics) was declared before those reruns, and only those two task ids may carry the flag. Both resealed: $\Delta_{\text{full}} + 0.530$ / $+0.505$. ARC-Easy stayed PARTIAL (unparseable $10/200 = 5\%$, metrics present) and entered the main sample; sensitivity (a) drops it.

Campaign close. BIRD and APPS stay out: `excluded for logistics, before any label existed`. The go threshold, the protocol file, and the honesty clause were not reopened after seeing AUC.

What the operator always owns. The following decisions never moved to the agent, even when the agent drafted the patch: freeze or not freeze the protocol file; keep or replace the 10×100 probe; the honesty clause and the go threshold; whether Mini-Dev may substitute for BIRD; the exact exclude sentence and the rule that it may be used only before a label exists; whether a PARTIAL with metrics (ARC-Easy) enters the sample; whether the pod may be killed; the disclosure boxes in this PDF. The agent owns packing, the unattended queue, harvest SHA checks, five-eye tables, and writing `PARTIAL` instead of a guessed Δ. When those two columns mixed — the SSH misreport, Gate 1 started early — the log names the mix rather than rewriting it.

2.5 Verification

Four checks sit under every sealed row, and a fifth sits under the paper's numbers.

Gates (automatic, do not block the queue): base unparseable $< 5\%$; pilot loss must fall; the systems block must be complete (torch, transformers, CUDA, driver, GPU name, base revision, seeds, `dry_run=false`); a greedy rerun of the unadapted base must match per-id `pass` (full 200 on list items 1 and every 10th; otherwise 30 rows; seed 20260820). Failure writes `STATUS=PARTIAL`. Smoke three-task five-eye was all green (GSM8K/WinoGrande unparseable $0/200$, Spider $1/200 = 0.5\%$; rerun mismatch 0).

Packages carry `MANIFEST.sha256`. Incremental harvest tars omit adapters and were SHA-checked against the worker; the full archive (adapters and isolated dirs included) has sha256 `71309f7c...f4028f7d`. Figures in Part 1 are redrawn from a frozen plot payload, not from a second analysis.

The analysis contract — go iff $\Delta_{\text{full}} > 0$, task-level LOTO logistic, Spearman on the two deltas, text-only control, bootstrap 95% CI, "AUC non-significant = publish negative" — was committed before the remaining 60 GPU slots. The coding agent drafted this Part 2 from that ledger and those reports. A human author has to walk every claim before the disclosure boxes below can be ticked; they are unchecked in this draft.

2.6 Significance of result

The qualifying result is a 61-row, execution-based ground-truth table that did not exist before the harness ran, plus a pre-registered reading of it: forging is the default (53/61), the 10-shot probe ranks gain, and the binary gate is not established. Small-model post-training spends real GPU time on tasks that look like GSM8K (negative transfer) and on tasks that look unimportant until a probe is actually run (description-only AUC below chance). A ranker that is honest about not being a stoplight is usable for budget allocation; a non-significant AUC hidden by post-hoc features would not be. The process claim, for this workshop, is narrower: a frozen protocol, a pre-registered honesty clause, and fail-closed logistics were enough to publish that negative gate rather than wait for more no-go tasks.

2.7 Interpretability

Protocol files, task packages, and run directories are append-only versioned (`protocol_v2`, `tasks_v3` leftover, `tasks_v4` 59-pack, `tasks_v5` bird-only). Old versions remain on disk; a run directory names the protocol and package it consumed. The ledger is append-only. Journal lines are stage-boundary events; Spider is the existence proof that resume does not replay S0–S3. Harvest is two-tier by design: incremental tars for monitoring (no adapters), one full tar for audit. Isolated failed dirs live inside that tar. A sealed run directory is meant to be readable without the agent: `task.json` and the package manifest, `journal.jsonl` stage boundaries, `eval_*_greedy.jsonl` for every labelled slice, `STATUS`, and `metrics.json` with the systems block (torch / transformers / CUDA / driver / GPU name / pinned base revision / seeds / `dry_run=false`). If that bundle is missing, the row is not a label. That is why BIRD and APPS are logistics excludes rather than no-go.

2.8 Limitations

The harness measures one base model, one LoRA recipe, greedy pass@1, and a $\Delta_{\text{full}} > 0$ gate that is empirically easy (8 no-go out of 61). Cluster AUCs were pre-declared and are not estimable. BIRD and APPS produced no label; we do not read that as "this model cannot learn SQL or competition-style code." Campaign GPU-hours were not summed and are not invented here. We report only walls that sit in sealed reports: smoke-task S0→S6 of GSM8K $\approx$ 2.16 h, WinoGrande $\approx$ 0.51 h, Spider $\approx$ 2.22 h effective (plus a killed half S4 $\approx$ 0.9 h not counted in the 2.22); a TDP-times-wall upper bound of $\approx$ 1.5 kWh on that three-task block, not a metered reading; serial production $\approx$ 39 h wall for 60 slots; Wave-1 TyDiQA $\approx$ 92 min and task419 $\approx$ 53 min inside the 3 h cap. The agent's connectivity misreport and the Gate 1-before-Gate 0 start are in the log because hiding them would make the STOP discipline look cleaner than it was.

3 Broader Impact

Guardrails earned their keep when they failed *closed*. The BIRD checker aborted on a missing `sha256` key rather than packing a substitute hash or writing a fake `metrics.json`; the later CUDA OOM likewise produced no label. GSM8K's negative Δ_{full} came with unparseable $0/200$ and a format-compliance shift from mixed `####`/last-number at base to `####` 200/200 after training — evidence against an EOS/truncation story, and the reason the operator could rule "format overwrite" instead of discarding a negative as a generation glitch. A rail that invents a number to keep a queue green is not a rail.

Pre-registration is what made the negative primary endpoint publishable. The honesty clause (bootstrap CI covering 0.5 equals a negative, to be printed) was committed before the remaining sixty GPU slots and before any AUC existed. After the seal, the tempting moves — add a feature, drop GSM8K, move the go threshold, reopen BIRD — were exactly the moves the contract forbids. With 8 no-go tasks the gate was always going to be under-powered; the clause prevented that fact from being negotiated away in the write-up.

STOP points were the human–agent interface, with a mixed record. The protocol-v3 STOP (keep 10×100; recitation is the probe) and the analysis STOP (no production before the SuperNI ids and the pre-reg were on the mainline) are the reason Part 1 has one recipe and one contract. The cost is real: Gate 1 started before Gate 0 was accepted, and the connectivity misreport looked like downtime until a human asked for a one-line multiplex check. A STOP that exists only as a chatbot apology does not stop a GPU.

Diagnosis was split on purpose. The agent is responsible for five-eye tables, SHA checks, journal resume, and for writing `PARTIAL` rather than a guessed Δ. The operator is responsible for mechanism rulings (format overwrite), for ops-fence vs measurement (APPS 3 h then 8 h), and for the exclude sentence that may be used only *before any label existed*. We would like the field to copy that split: freeze the recipe, pre-register the honesty clause, fail closed, and keep the operator on the one-way door out of the sample.

A Reproducibility Statement

The protocol file, pre-registration, and the sealed 61-row per-task table are packed in the supplementary archive. Protocol hyperparameters were never edited after freeze. Dataset papers for the literature-layer table: Disclosed run-level exceptions (OOM fallback on two tasks; one PARTIAL in-sample row) are listed in Setup and in the supplementary protocol notes. Figures in Part 1 are redrawn from the frozen plot payload; they are not a new analysis.

B Disclosure Statement

Agent(s) / model(s) used The 61-row experimental table was produced by a frozen Python harness on a single serial GPU queue (protocol v2; base model Qwen2.5-1.5B-Instruct at the pinned revision in the protocol file). An LLM coding agent packed tasks, monitored the queue, diagnosed failures, executed the pre-registered analysis, and drafted Parts 1–3 of this manuscript, including figures generated from the frozen plot payload. The coding agent's model identity is omitted here to preserve double-blind review and will be named to the program chairs. No experimental number in Part 1 was computed outside the sealed analysis script. Human status of this draft: data, hashes, and analysis artifacts were produced under the harness and the pre-registration; this text has *not* yet been line-checked by a human author. The two attestations below are therefore left unchecked.

- ☐ A human author has curated this work and verified every claim, figure, methodology detail, and reference.
- ☐ A human author takes full accountability for the submission as a human-authored, human-verified artifact.

References

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.

Yonatan Bisk, Rowan Zellers, Ronan Le Bras, Jianfeng Gao, and Yejin Choi. PIQA: Reasoning about physical commonsense in natural language. *arXiv preprint arXiv:1911.11641*, 2019.

Jonathan H. Clark, Eunsol Choi, Michael Collins, Dan Garrette, Tom Kwiatkowski, Vitaly Nikolaev, and Jennimaria Palomaki. TyDi QA: A benchmark for information-seeking question answering in typologically diverse languages. *arXiv preprint arXiv:2003.05002*, 2020.

Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try ARC, the AI2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

Dheeru Dua, Yizhong Wang, Pradeep Dasigi, Gabriel Stanovsky, Sameer Singh, and Matt Gardner. DROP: A reading comprehension benchmark requiring discrete reasoning over paragraphs. *arXiv preprint arXiv:1903.00161*, 2019.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. *arXiv preprint arXiv:2103.03874*, 2021.

Haoyang Li, Jing Zhang, Hanbing Liu, Ju Fan, Xiaokang Zhang, Jun Zhu, Renjie Wei, Hongyan Pan, Cuiping Li, and Hong Chen. CodeS: Towards building open-source language models for text-to-SQL. *arXiv preprint arXiv:2402.16347*, 2024.

Haowei Lin, Baizhou Huang, Haotian Ye, Qinyu Chen, Zihao Wang, Sujian Li, Jianzhu Ma, Xiaojun Wan, James Zou, and Yitao Liang. Selecting large language model to fine-tune via rectified scaling law. *arXiv preprint arXiv:2402.02314*, 2024.

Rahul Nair and Chun Tao. The fine-tuning trap: Evaluating negative transfer and the role of PEFT in sub-1B mathematical reasoning. *arXiv preprint arXiv:2606.06920*, 2026.

Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. WinoGrande: An adversarial winograd schema challenge at scale. *arXiv preprint arXiv:1907.10641*, 2019.

Yizhong Wang, Swaroop Mishra, Pegah Alipoormolabashi, Yeganeh Kordi, Amirreza Mirzaei, et al. Super-naturalinstructions: Generalization via declarative instructions on 1600+ NLP tasks. *arXiv preprint arXiv:2204.07705*, 2022.

Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, Zilin Zhang, and Dragomir Radev. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task. *arXiv preprint arXiv:1809.08887*, 2018.

Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. HellaSwag: Can a machine really finish your sentence? *arXiv preprint arXiv:1905.07830*, 2019.